CUSTOMER CHURN PREDICTION

Predictive Analytics & AI Model Deployment

Internship Project Report
Artificial Intelligence & Machine Learning
Cognevance Technologies

Project Level: Level 3 – Advanced

Repository: github.com/shaawze/cognevance_CustomerChurnPredict

Prepared by: Subiksha Sri.M

Abstract

This project develops a machine-learning workflow to estimate whether a telecommunications customer is likely to discontinue service. Using the IBM Telco Customer Churn dataset, the work covers data inspection, cleaning, exploratory analysis, feature preprocessing, model training and comparison, decision-threshold selection, and local deployment through a Streamlit interface. Logistic Regression was selected for deployment because it provided stronger churn recall and ROC-AUC than the evaluated Random Forest model, despite lower overall accuracy.

1. Introduction

Customer churn is the loss of customers who stop using a company's products or services. Identifying customers who may leave can help an organization prioritize retention outreach and investigate service or pricing issues. This project frames churn prediction as supervised binary classification using customer account, service, contract, tenure, and billing attributes.

2. Objectives

The objectives are to inspect and prepare telecom customer data; explore churn patterns; build preprocessing and classification pipelines; train and compare Logistic Regression and Random Forest; evaluate classification metrics; tune the probability threshold; and deploy predictions through Streamlit.

3. Dataset

The project uses the IBM Telco Customer Churn dataset. The notebook reports 7,043 rows and 21 columns. The target is Churn. Input columns cover customer attributes, subscribed services, contract information, and charges. The customerID identifier is excluded from the predictive features, leaving 19 input features.

4. Data Preparation

The notebook checks duplicates, blank values, data types, and target distribution. TotalCharges is converted from text to numeric; 11 missing values are imputed using the median. Numerical features are standardized with StandardScaler. Categorical features are one-hot encoded with OneHotEncoder(handle_unknown='ignore'). The data is split 80:20 using random_state=42 and stratification. The feature design uses four numerical and fifteen categorical features.

5. Exploratory Data Analysis

The analysis examines churn distribution and churn relationships with contract type, tenure, and monthly charges. Contract-level churn rates recorded in the notebook are: month-to-month 42.71%, one year 11.27%, and two year 2.83%. Month-to-month customers show a higher churn rate than customers on longer contracts. These are descriptive associations, not proof of causation.

6. Modeling Methodology

Logistic Regression is trained in a preprocessing pipeline using max_iter=1000 and class_weight='balanced'. A Random Forest comparison model uses 300 estimators, random_state=42, class_weight='balanced', and parallel execution.

7. Model Evaluation

Accuracy measures overall correct classifications; churn precision measures the reliability of positive churn predictions; churn recall measures the proportion of actual churn cases identified; F1 is the harmonic mean of precision and recall; ROC-AUC measures ranking discrimination across thresholds. Random Forest achieved higher accuracy and churn precision, while Logistic Regression achieved higher churn recall and ROC-AUC. Logistic Regression was selected because identifying more potentially at-risk customers was the project's priority.

Metric	Logistic Regression	Random Forest
Accuracy	73.81%	78.21%
ROC-AUC	0.841	0.822
Churn precision	50.4%	62%
Churn recall	78.3%	47%
Churn F1-score	61.4%	53%

8. Decision Threshold Tuning

The deployed Logistic Regression model uses a probability threshold of 0.45. At this threshold, recorded churn-class metrics are approximately 48.3% precision, 83.7% recall, and 61.3% F1-score. A lower threshold flags more customers but may increase false-positive alerts.

Threshold	Precision	Recall	F1
0.30	42.9%	92.8%	58.7%
0.35	44.8%	90.6%	59.9%
0.40	46.6%	86.6%	60.6%
0.45	48.3%	83.7%	61.3%
0.50	50.4%	78.3%	61.4%

9. Deployment

The selected pipeline is serialized with Joblib in final_churn_model.pkl together with the 0.45 threshold and model name. The Streamlit app.py interface collects customer attributes, loads the saved bundle, calculates churn probability, compares it with the threshold, and displays the prediction. A local test displayed 65.30% churn probability, above the 45% threshold, resulting in 'Likely to churn'. This sample demonstrates app behavior; it is not a model accuracy measure.

10. Project Structure

The repository contains Customer_Churn_Prediction.ipynb (analysis and training), Telco-Customer-Churn.csv (dataset), app.py (Streamlit interface), final_churn_model.pkl (trained pipeline and threshold), requirements.txt (dependencies), README.md (project documentation), and screenshots/ (application evidence).

11. Technologies Used

Python; Pandas; NumPy; Matplotlib; Seaborn; Scikit-learn; Joblib; Streamlit; Google Colab.

12. Limitations and Responsible Use

The model is trained and evaluated on one historical telecom dataset; performance may differ on newer or different customer populations. Class imbalance means accuracy alone is insufficient. The 0.45 threshold may generate false-positive alerts. Predictions are estimates, not guarantees or explanations of individual behavior. Associations do not establish causation. Operational use requires validation on current data, monitoring, and fairness review.

13. Conclusion

The project demonstrates an end-to-end predictive analytics workflow from data preparation and exploratory analysis to model comparison, threshold tuning, serialization, and Streamlit deployment. Logistic Regression was selected because it achieved stronger churn recall and ROC-AUC than Random Forest in the recorded evaluation.

14. Repository

GitHub repository: https://github.com/shaawze/cognevance_CustomerChurnPrediction

Appendix – Running the Application

Install dependencies using pip install -r requirements.txt. Start the application using streamlit run app.py. Keep final_churn_model.pkl in the same directory as app.py.